Universidad de las Américas Puebla

Escuela de ingeniería

Departamento de computación, electrónica y mecatrónica

UDLAP®

Lab report #6

Course: Digital design LRT2022-7

Equipo: 2

183339 Juan Pablo Lopez Moreno LMT
185406 Amy Marianee Ramírez Sánchez LMT

November 26th, 25, San Andrés Cholula, Puebla

1 Abstract

This report will present the procedure utilized during our lab experimentation, for the sixth lab report. The practice ensured that we continued developing the usage of FPGAs, as well as, continuing the use of VHDL as a programming language. The practice focused on the development and testing of flip flops, ensuring the understanding of these, and the implmenetation of these components using a clock.

2 Introduction

Flip-flops are pivotal components in digital electronics, serving as fundamental units for storing binary data. They are essential in sequential logic circuits where information needs to be stored, processed, and outputted in a controlled manner. [1] Components like flip-flops are the staples of memory elements, being some of the most frequently known uses for these their appearance in registers, counters, and memory devices. Flip-flops are bistable devices, meaning they have two stable states, typically represented as 0 and 1. They can maintain their state indefinitely until triggered to change by an input signal, making them ideal for storing bits of information. These Flip flops can be classified into several types, however, for the practical developmnet in the lab, we will only be focusing on the following variation:

- SR Flip-Flop: The Set-Reset flip-flop has two inputs, S (set) and R (reset), and can store one bit of data. It toggles between two stable states based on the input signals
- D Flip-Flop: The Data flip-flop captures and holds the value of its D (data) input when triggered by a clock signal. It is widely used for synchronizing data transfers.
- JK Flip-Flop: Named after its inventors, Jack Kilby and Jerry Merryman, the JK flip-flop combines the features of the SR and T flip-flops, offering more flexibility in circuit design.
- T Flip-Flop: The Toggle flip-flop changes its state (either 0 or 1) with each clock pulse, effectively toggling between its two states.

[1] The very relevance of these components in digital systems have been fundamnetal in the development of technologies for many years, the reason being, the capabilities for storing data, maquing sequential systems possible.

3 Objectives

The objectives of this lab are:

- Program and simulate Flip-flops in VHDL
- Implement Synchronous FF on Basys 3 board.

For the purpose of this lab, we must understand the following concepts:

- Sequential Logic Circuits

- VHDL Programming
- FPGA Implementation

Of which, we must begin explaining A sequential logic circuit is a type of digital circuit that can store and remember information or data between clock cycles. It uses a memory element, such as a flip-flop or register, to store and update the state of the circuit based on the input and previous state. These circuits can perform a variety of functions, such as counting, shifting, and detecting and responding to specific input patterns. [2] VHDL, which has been used in previous labs, is a hardware description language that allows designers to describe the behavior and structure of digital systems. VHDL is widely used for designing FPGAs and ASICs (Application-Specific Integrated Circuits) due to its ability to model complex digital systems at various levels of abstraction.[3] Finally,field programmable gate array (FPGA) is a versatile type of integrated circuit, which, unlike traditional logic devices such as application-specific integrated circuits (ASICs), is designed to be programmable (and often reprogrammable) to suit different purposes, notably high-performance computing (HPC) and prototyping.[4] FPGA implementation involves programming the FPGA device with the designed VHDL code to realize the desired combinational logic circuit. This process includes synthesis, mapping, placement, and routing of the design onto the FPGA fabric.

4 Methodology

The methodology applied in this laboratory practice was organized into three main phases: logical analysis, simulation in EDA Playground, and physical implementation on the Basys 3 board.

4.1 Logical Analysis Phase

During this initial stage, the following activities were carried out:

- The truth tables and characteristic equations of the synchronous RS, JK, D, and T flip-flops were reviewed, identifying their hold conditions, state-change conditions, and invalid cases.
- The differences between asynchronous and synchronous operations were analyzed, emphasizing that all flip-flops used in this practice depend exclusively on the rising edge of the clock signal.
- The expected behavior of each flip-flop was defined based on its state-transition table, establishing verification criteria for the subsequent simulations.

4.2 EDA Playground Simulation Phase

In the EDA Playground digital simulation environment, the following activities were performed:

- The synchronous models of the RS, JK, D, and T flip-flops were programmed in VHDL using processes sensitive to the rising edge of the clock and internal registers to store the state.

- Independent testbenches were developed for each flip-flop, including clock-signal generation and the application of specific stimulus sequences.
- Simulations were executed and the resulting waveforms were analyzed, verifying that the output Q changed only on rising clock edges and matched the expected theoretical behavior.

4.3 Basys 3 Board Implementation Phase

Finally, the physical implementation of the synchronous flip-flops was carried out on the Basys 3 board following the procedure described below:

- The VHDL code for each flip-flop was synthesized, and the corresponding configuration files (bitstreams) were generated.
- Pin constraints (in the `.xdc` file) were assigned to map:
  - Control inputs (S, R, J, K, D, T) to the switches,
  - The clock signal to the 100 MHz onboard oscillator,
  - The output Q to the Basys 3 LEDs.
- The designs were then loaded onto the FPGA, tests were performed by manipulating the switches, and it was verified that the observed behavior matched both the theoretical models and the simulation results.

5 Results

In this lab, the physical circuits were not built, but based on certain truth tables, EDAWaves were generated in order to accomplish every single sequential circuit given:

5.1 RS Flip-Flop

For the RS flip-flop, the following truth table was used to generate the EDA Code:

Table 1: RS Flip-Flop Truth Table

Inputs		**Outputs**		
S	**R**	$\mathbf{Q}_{n+1}$	$\mathbf{Q}'_{n+1}$	**Function**
0	0	$\mathbf{Q}_n$	$\mathbf{Q}'_n$	**Hold (No Change)**
0	1	0	1	**Reset**
1	0	1	0	**Set**
1	1	×	×	**Invalid (Forbidden)**

Based on this truth table, the following Code was generated in EDA Playgrounds:

Código 1: Code 1: For exercise 1

```
library IEEE;
use ieee.std_logic_1164.all;

entity sr_sync is
    port(
        clk : in std_logic;
        s   : in std_logic;
        r   : in std_logic;
        Q   : out std_logic
    );
end sr_sync;

architecture behavioral of sr_sync is
    signal Q_reg : std_logic := '0';
begin
    process(clk)
    begin
        if rising_edge(clk) then
            if (s = '0' and r = '0') then
                Q_reg <= Q_reg;
            elsif (s = '1' and r = '0') then
                Q_reg <= '1';
            elsif (s = '0' and r = '1') then
                Q_reg <= '0';
            elsif (s = '1' and r = '1') then
                Q_reg <= 'X';
            end if;
        end if;
    end process;

    Q <= Q_reg;
end behavioral;
```

With it's appropriate testbench:

Código 2: Code 2: Testbench for exercise 1

```
library IEEE;
use ieee.std_logic_1164.all;

entity tb_sr_sync is
end tb_sr_sync;

architecture behavior of tb_sr_sync is

    component sr_sync
```

```
        port(
            clk : in std_logic;
            s   : in std_logic;
            r   : in std_logic;
            Q   : out std_logic
        );
    end component;

    signal clk : std_logic := '0';
    signal s   : std_logic := '0';
    signal r   : std_logic := '0';
    signal Q   : std_logic;

    constant clk_period : time := 10 ns;

begin

    uut : sr_sync
        port map(
            clk => clk,
            s   => s,
            r   => r,
            Q   => Q
        );

    -- Generador de reloj
    clk_process : process
    begin
        clk <= '0';
        wait for clk_period/2;
        clk <= '1';
        wait for clk_period/2;
    end process;

    stim_proc : process
    begin
        -- Test 1: Retener
        s <= '0'; r <= '0';
        wait for clk_period;

        -- Test 2: Reset
        s <= '0'; r <= '1';
        wait for clk_period;

        -- Test 3: Retener
```

```
        s <= '0'; r <= '0';
        wait for clk_period;

        -- Test 4: Set
        s <= '1'; r <= '0';
        wait for clk_period;

        -- Test 5: Retener
        s <= '0'; r <= '0';
        wait for clk_period;

        s <= '1'; r <= '1';
        wait for clk_period;

        wait;
    end process;

end behavior;
```

And, after developing the code in EDA Playgrounds, the following Wave was generating, showcasing the values of the function:

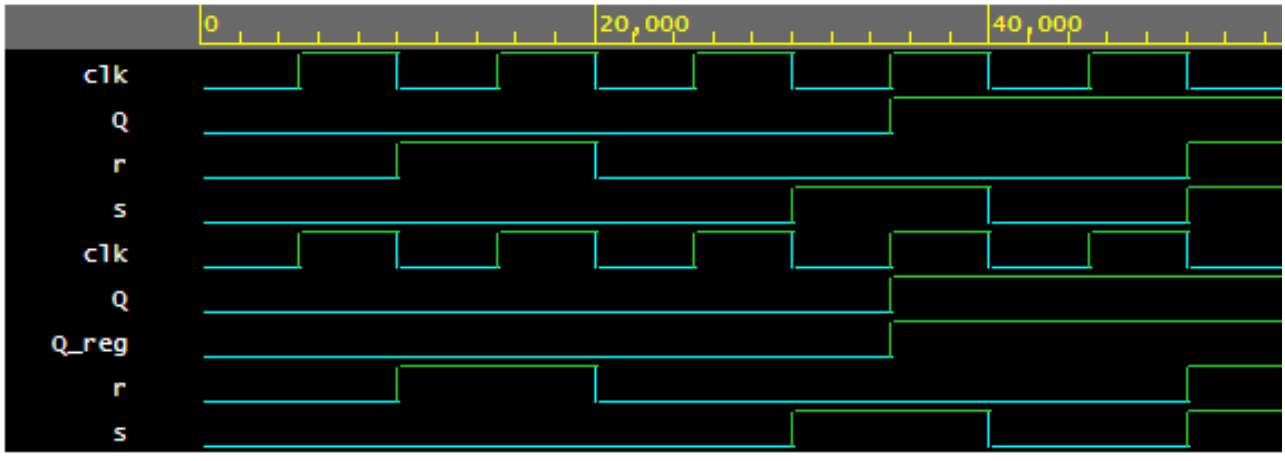

Figure 1: Wave generated for the RS flip-flop.

Finally, the code was synthezised and implemented in the Basys 3 board, obtaining the expected results.

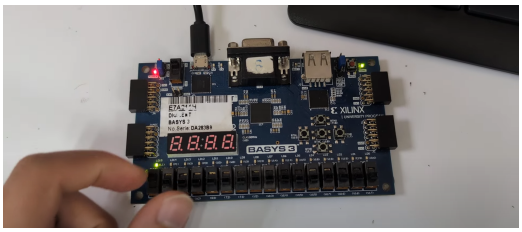

Figure 2: Evidence of the functioning of the RS flip-flop in the Basys 3 board.

5.2 JK Flip-Flop

For the JK flip-flop, the following truth table was used to generate the EDA Code:

Table 2: JK Flip-Flop Truth Table

Inputs		**Outputs**		
J	**K**	$\mathbf{Q}_{n+1}$	$\mathbf{Q}'_{n+1}$	**Function**
0	0	$\mathbf{Q}_n$	$\mathbf{Q}'_n$	**Hold (No Change)**
0	1	0	1	**Reset**
1	0	1	0	**Set**
1	1	$\mathbf{Q}'_n$	$\mathbf{Q}_n$	**Toggle**

Based on this truth table, the following Code was generated in EDA Playgrounds:

Código 3: Code 1: For exercise 2

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity jk_flipflop is
    Port (
        clk : in  STD_LOGIC;
        J   : in  STD_LOGIC;
        K   : in  STD_LOGIC;
        Q   : out STD_LOGIC
    );
end jk_flipflop;

architecture Behavioral of jk_flipflop is
    signal q_int : STD_LOGIC := '0';
begin

    process(clk)
    begin
        if rising_edge(clk) then
            case (J & K) is
                when "00" => q_int <= q_int;          -- No cambia
```

```
                when "01" => q_int <= '0';          -- Reset
                when "10" => q_int <= '1';          -- Set
                when "11" => q_int <= not q_int;    -- Toggle
                when others => null;
            end case;
        end if;
    end process;

    Q <= q_int;

end Behavioral;
```

With it's appropriate testbench:

Código 4: Code 2: Testbench for exercise 2

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_jkff is
end tb_jkff;

architecture Behavioral of tb_jkff is

    signal clk : STD_LOGIC := '0';
    signal J   : STD_LOGIC := '0';
    signal K   : STD_LOGIC := '0';
    signal Q   : STD_LOGIC;

    -- Instancia del JK
    component jk_flipflop
        Port (
            clk : in  STD_LOGIC;
            J   : in  STD_LOGIC;
            K   : in  STD_LOGIC;
            Q   : out STD_LOGIC
        );
    end component;

begin

    uut: jk_flipflop
        port map (
            clk => clk,
            J   => J,
            K   => K,
            Q   => Q
```

```
        );

    -- Clock 10 ns
    clk <= not clk after 5 ns;

    stim_proc: process
    begin
        -- 00: Hold
        J <= '0'; K <= '0';
        wait for 20 ns;

        -- 10: Set
        J <= '1'; K <= '0';
        wait for 20 ns;

        -- 01: Reset
        J <= '0'; K <= '1';
        wait for 20 ns;

        -- 11: Toggle
        J <= '1'; K <= '1';
        wait for 40 ns;

        wait;
    end process;

end Behavioral;
```

And, after developing the code in EDA Playgrounds, the following Wave was generating, showcasing the values of the function:

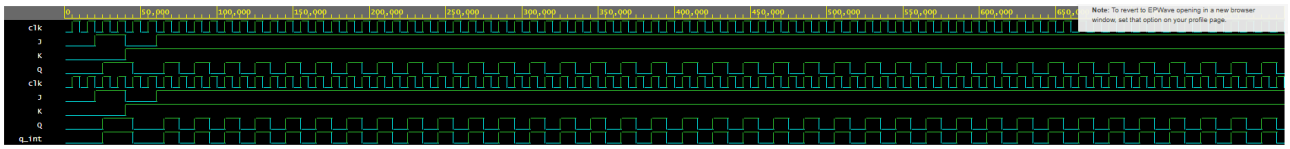

Figure 3: Wave generated for the JK flip-flop.

Finally, the code was synthesized and implemented in the Basys 3 board, obtaining the expected results.

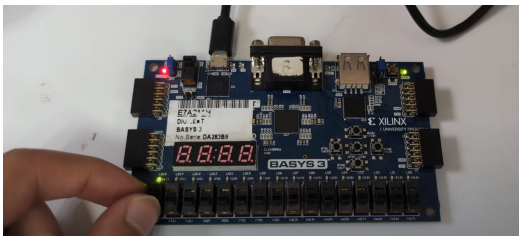

Figure 4: Evidence of the functioning of the JK flip-flop in the Basys 3 board.

5.3 D Flip-Flop

For the D flip-flop , the following truth table was used to generate the EDA Code:

Table 3: D Flip-Flop Truth Table

Input	**Outputs**		
D	$\mathbf{Q}_{n+1}$	$\mathbf{Q}'_{n+1}$	**Function**
0	0	1	**Reset (Clear)**
1	1	0	**Set**

Based on this truth table, the following Code was generated in EDA Playgrounds:

Código 5: Code 1: For exercise 3

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity d_flipflop is
    port(
        clk : in std_logic;
        d   : in std_logic;
        q   : out std_logic
    );
end d_flipflop;

architecture behavioral of d_flipflop is
    signal q_int : std_logic := '0';
begin
    process(clk)
    begin
        if clk = '1' then          -- cuando clk=1
            q_int <= d;            -- Q(n+1) = D
        end if;                    -- cuando clk=0, no cambia
    end process;

    q <= q_int;
end behavioral;
```

With it's appropriate testbench:

Código 6: Code 2: Testbench for exercise 3

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_dff is
end tb_dff;

architecture behavior of tb_dff is

    component d_flipflop
        port(
            clk : in std_logic;
            d   : in std_logic;
            q   : out std_logic
        );
    end component;

    signal clk : std_logic := '0';
    signal d   : std_logic := '0';
    signal q   : std_logic;

begin

    uut: d_flipflop
        port map(
            clk => clk,
            d   => d,
            q   => q
        );

    -- reloj
    clk_process : process
    begin
        clk <= '0';
        wait for 5 ns;
        clk <= '1';
        wait for 5 ns;
    end process;

    stim_proc : process
    begin
        d <= '1';
        wait for 10 ns;     -- Q se mantiene
```

```
        d <= '0';
        wait for 10 ns;

        d <= '1';
        wait for 10 ns;

        wait;
    end process;

end behavior;
```

And, after developing the code in EDA Playgrounds, the following Wave was generating, showcasing the values of the function:

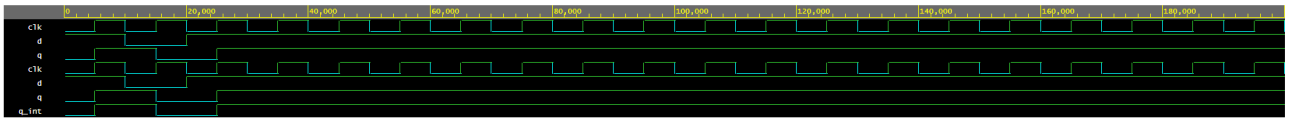

Figure 5: Wave generated for the D flip-flop.

Finally, the code was synthezised and implemented in the Basys 3 board, obtaining the expected results.

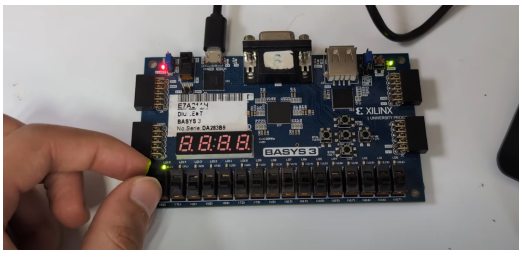

Figure 6: Evidence of the functioning of the D flip-flop in the Basys 3 board.

5.4 T Flip-Flop

For the T flip-flop, the following truth table was used to generate the EDA Code:

Table 4: T Flip-Flop Truth Table

Input	**Outputs**		
T	$\mathbf{Q}_{n+1}$	$\mathbf{Q}'_{n+1}$	**Function**
0	$\mathbf{Q}_n$	$\mathbf{Q}'_n$	**Hold (No Change)**
1	$\mathbf{Q}'_n$	$\mathbf{Q}_n$	**Toggle**

Based on this truth table, the following Code was generated in EDA Playgrounds:

Código 7: Code 1: For exercise 1

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;

entity t_flipflop is
    port(
        clk : in std_logic;
        t   : in std_logic;
        q   : out std_logic
    );
end t_flipflop;

architecture behavioral of t_flipflop is
    signal q_int : std_logic := '0';
begin

    process(clk)
    begin
        if clk = '1' then           -- solo cambia cuando clk = 1
            if t = '1' then
                q_int <= not q_int;
            end if;
            -- si T=0 no cambia
        end if;
        -- si clk=0 no cambia
    end process;

    q <= q_int;

end behavioral;
```

With it's appropriate testbench:

Código 8: Code 2: Testbench for exercise 1

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_tff is
end tb_tff;

architecture behavior of tb_tff is

    component t_flipflop
        port(
            clk : in std_logic;
            t   : in std_logic;
            q   : out std_logic
        );
```

```
    end component;

    signal clk : std_logic := '0';
    signal t   : std_logic := '0';
    signal q   : std_logic;

begin

    uut: t_flipflop
        port map(
            clk => clk,
            t   => t,
            q   => q
        );

    -- Generador de reloj
    clk_process : process
    begin
        clk <= '0';
        wait for 5 ns;
        clk <= '1';
        wait for 5 ns;
    end process;

    stim_proc : process
    begin
        -- CLK=0 (no cambia)
        t <= '0';
        wait for 10 ns;

        -- T=0 (mantiene)
        t <= '0';
        wait for 10 ns;

        -- T=1 (toggle)
        t <= '1';
        wait for 10 ns;

        -- Otro toggle
        t <= '1';
        wait for 10 ns;

        wait;
    end process;
```

```
end behavior;
```

And, after developing the code in EDA Playgrounds, the following Wave was generating, showcasing the values of the function:

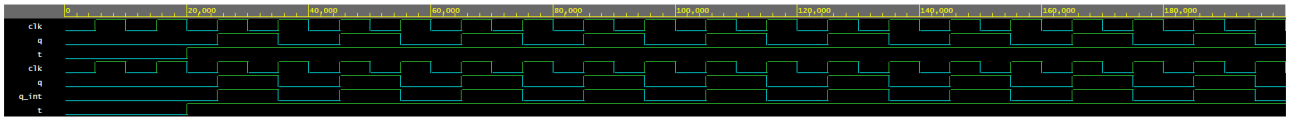

Figure 7: Wave generated for the T flip-flop.

Finally, the code was synthezised and implemented in the Basys 3 board, obtaining the expected results.

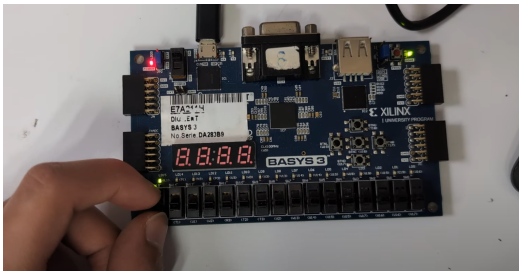

Figure 8: Evidence of the functioning of the T flip-flop in the Basys 3 board.

6 Analysis of Results

In this laboratory practice, four fundamental sequential elements were analyzed—the synchronous RS, JK, D, and T flip-flops—following three main stages: logical analysis of theoretical behavior, digital simulation in EDA Playground using VHDL, and practical verification on the Basys 3 board.

The observed behavior for each flip-flop is described in detail below.

6.1 RS Flip-Flop

6.1.1 Logical Analysis

The synchronous RS flip-flop has two inputs: S (set) and R (reset), as well as a stored output Q. Its theoretical behavior confirms:

- $S = 1$ and $R = 0$: The output is set to 1 at the rising edge of the clock.
- $S = 0$ and $R = 1$: The output resets to 0.
- $S = 0$ and $R = 0$: The flip-flop preserves its previous state.
- $S = 1$ and $R = 1$: This is an invalid condition due to contradictory commands, so it was avoided during testing.

This analysis reinforces the RS flip-flop as a clock-controlled storage element.

6.1.2 Analysis in EDA Playground

The simulations showed:

- Transitions of Q only at the positive clock edge.
- Correct changes in state according to valid S and R combinations.
- Proper conservation of the state when both inputs were 0.
- Absence of the invalid condition due to the testbench design.

No glitches or asynchronous behaviors were observed, confirming a correct VHDL implementation.

6.1.3 Implementation on Basys 3

On the Basys 3 board:

- Switches were assigned to inputs S and R.
- Output Q was displayed through an LED.

The practical results matched the theory:

- Activating S turned the LED on.
- Activating R turned the LED off.
- Both inputs at 0 preserved the previous value.

This verified correct synthesis, pin assignment, and synchronous behavior of the RS flip-flop.

6.2 JK Flip-Flop

6.2.1 Logical Analysis

The JK flip-flop is an improved version of the RS type, as it eliminates its invalid state. Its behavior is:

- $J = 0, K = 0$: Q retains its state.
- $J = 1, K = 0$: Q is set to 1.
- $J = 0, K = 1$: Q resets to 0.
- $J = 1, K = 1$: Q toggles between 0 and 1 on each rising clock edge.

This makes it one of the most versatile flip-flops in sequential design.

6.2.2 Analysis in EDA Playground

The simulation waveforms showed:

- Proper state retention when $J = K = 0$.
- Accurate set and reset responses according to the inputs.
- Clean toggling when J and K were both 1.
- Changes strictly at the rising clock edge.

The absence of undefined states demonstrates a valid model.

6.2.3 Implementation on Basys 3

During physical implementation:

- Switches controlled J and K.
- A LED represented output Q.
- The device correctly exhibited set, reset, and toggle functions.
- Its behavior matched both the simulation and the theoretical analysis, ensuring correct design and pin mapping.

6.3 D Flip-Flop

6.3.1 Logical Analysis

The D flip-flop simplifies storage to a single input, D, with the following behavior:

- At every rising clock edge, Q adopts the value of D.
- Between edges, Q remains constant.

It is the most widely used flip-flop for retaining and synchronizing data.

6.3.2 Analysis in EDA Playground

The simulation confirmed:

- The output updated immediately on each rising clock edge.
- Changes in D outside of the clock edge did not affect Q.
- No intermediate values or unexpected artifacts appeared.

The waveforms validated the ideal operation of a D-type flip-flop.

6.3.3 Implementation on Basys 3

With D connected to a switch and Q to an LED, it was verified that:

- The LED changed exactly when the clock produced a rising edge.
- Changes in D between edges did not alter the output.

The behavior fully matched the theoretical and simulated results, confirming correct synchronization and data propagation.

6.4 T Flip-Flop

6.4.1 Logical Analysis

The T flip-flop uses a single input, T, with the following behaviors:

- $T = 0$: The output remains unchanged.
- $T = 1$: The output toggles its state on each rising edge.

It is widely used in frequency dividers and binary counters.

6.4.2 Analysis in EDA Playground

The simulations showed:

- State retention when $T = 0$.
- Precise and error-free toggling when $T = 1$.
- Changes occurring only at the positive clock edge.
- Full stability without glitches or undesirable behavior.

This validated correct implementation of the toggle mechanism.

6.4.3 Implementation on Basys 3

During physical testing:

- A switch controlled T.
- A LED displayed output Q.
- With $T = 1$, the LED alternated its state at each clock pulse.
- With $T = 0$, the LED remained stable.

Observations fully matched the simulated and theoretical results.

7 Conclusion

This laboratory practice enabled the analysis, simulation, and physical implementation of four fundamental sequential circuits: the synchronous RS, JK, D, and T flip-flops. By comparing theoretical behavior, VHDL simulations in EDA Playground, and hardware verification on the Basys 3 board, it was confirmed that each flip-flop operated according to its intended design and characteristic equations. The analysis of state tables and transition behaviors provided a solid understanding of the logical operation of each device, highlighting their roles in memory storage, synchronization, and state control. Digital simulation validated the correctness of the VHDL implementations, showing clean transitions at the rising clock edge and the absence of glitches or asynchronous behavior. The Basys 3 implementation reproduced these results physically, with switches and LEDs clearly demonstrating state retention, setting, resetting, and toggling as expected. In conclusion, this practice reinforced the understanding of essential sequential logic principles and the importance of verifying digital designs both through simulation and hardware implementation. Additionally, it strengthened skills in using tools such as EDA Playground and FPGA development boards, which are crucial for designing reliable, synchronous digital systems used in counters, registers, and more complex sequential architectures.

References

[1] M. Mathiarasu. (2024) Flip-flops in electronics. [En línea]. Disponible: https://www.linkedin.com/pulse/flip-flops-electronics-mathiarasu-m-po8oc/

[2] ChipVerify. (2024) Sequential logic. [En línea]. Disponible: https://www.chipverify.com/digital-fundamentals/sequential-logic

[3] Intel Corporation, "Vhdl definition," https://www.intel.com/content/www/us/en/programmable/quartushelp/17.0/reference/glossary/def_vhdl.htm, 2017, accessed: 2024-10-27.

[4] J. Schneider and I. Smalley. (2025) Field programmable gate array. [En línea]. Disponible: https://www.ibm.com/think/topics/field-programmable-gate-arrays